

Contents

1	Tools and Frameworks	2
1.1	Comprehensive Software Stack for AI-IDS Development	2
1.2	Table of Contents	2
1.3	1. Deep Learning Frameworks	2
1.3.1	1.1 TensorFlow	2
1.3.2	1.2 PyTorch	3
1.3.3	1.3 Keras	3
1.3.4	1.4 ONNX Runtime	3
1.4	2. Classical Machine Learning	4
1.4.1	2.1 Scikit-learn	4
1.4.2	2.2 XGBoost	4
1.4.3	2.3 LightGBM	4
1.4.4	2.4 CatBoost	4
1.5	3. Network and SDN Tools	5
1.5.1	3.1 ONOS (Open Network Operating System)	5
1.5.2	3.2 OpenDaylight	5
1.5.3	3.3 Ryu	5
1.5.4	3.4 Mininet	6
1.5.5	3.5 NS-3 (Network Simulator 3)	7
1.5.6	3.6 OMNET++	7
1.6	4. Data Processing and Analysis	7
1.6.1	4.1 Pandas	7
1.6.2	4.2 NumPy	7
1.6.3	4.3 Apache Spark (PySpark)	8
1.6.4	4.4 Dask	8
1.6.5	4.5 Scapy	8
1.7	5. Explainable AI (XAI) Libraries	8
1.7.1	5.1 SHAP	8
1.7.2	5.2 LIME	9
1.7.3	5.3 Captum	9
1.8	6. Distributed Computing and Federated Learning	9
1.8.1	6.1 TensorFlow Federated	9
1.8.2	6.2 PySyft	10
1.8.3	6.3 Flower (Flower)	10
1.9	7. Network Simulation and Testbed	10
1.9.1	7.1 tcpdump	10
1.9.2	7.2 Wireshark/tshark	10
1.9.3	7.3 CICFlowMeter	11
1.10	8. Containerization and Orchestration	11
1.10.18.1	Docker	11
1.10.28.2	Kubernetes	11
1.11	9. Monitoring and Visualization	11
1.11.19.1	Matplotlib	11
1.11.29.2	Seaborn	11
1.11.39.3	Plotly	12
1.11.49.4	Grafana	12

1.11.59.5 Prometheus	12
1.11.69.6 ELK Stack (Elasticsearch, Logstash, Kibana)	12
1.1210. Development Tools	13
1.12.110.1 Jupyter Notebook/Lab	13
1.12.210.2 VS Code	13
1.12.310.3 PyCharm	13
1.1311. Version Control and Collaboration	13
1.13.111.1 Git	13
1.13.211.2 GitHub	13
1.13.311.3 DVC (Data Version Control)	14
1.14 Summary: Complete Environment Setup	14
1.14.1 Installation Script	14
1.15 Conclusion	15

1 Tools and Frameworks

1.1 Comprehensive Software Stack for AI-IDS Development

Complete Technology Stack and Tools

Last Updated: December 22, 2025

1.2 Table of Contents

1. Deep Learning Frameworks
 2. Classical Machine Learning
 3. Network and SDN Tools
 4. Data Processing and Analysis
 5. Explainable AI (XAI) Libraries
 6. Distributed Computing and Federated Learning
 7. Network Simulation and Testbed
 8. Containerization and Orchestration
 9. Monitoring and Visualization
 10. Development Tools
 11. Version Control and Collaboration
-

1.3 1. Deep Learning Frameworks

1.3.1 1.1 TensorFlow

Version: 2.13+ (Latest stable)

Purpose: Primary deep learning framework for model development

Usage: - CNN, LSTM, Transformer implementation - Model training and inference - TensorFlow Lite for edge deployment - TensorBoard for visualization

Installation:

```
pip install tensorflow==2.13.0 tensorflow-gpu==2.13.0
```

Key APIs: - tf.keras: High-level API for model building - tf.data: Efficient data pipeline - tf.distribute: Distributed training - tf.lite: Model conversion for edge devices

1.3.2 1.2 PyTorch

Version: 2.0+ (Latest stable)

Purpose: Alternative deep learning framework, preferred for research

Usage: - Flexible model prototyping - Custom loss functions and layers - ONNX export for interoperability - Dynamic computational graphs

Installation:

```
pip install torch==2.0.1 torchvision==0.15.2 torchaudio==2.0.2
```

Key Libraries: - torch.nn: Neural network modules - torch.optim: Optimization algorithms - torchvision: Computer vision utilities - torch.distributed: Distributed training

1.3.3 1.3 Keras

Version: Included with TensorFlow 2.x

Purpose: High-level neural networks API

Usage: - Rapid prototyping - Sequential and Functional API - Transfer learning - Pre-trained models

Key Features: - User-friendly - Modular and composable - Backend-agnostic (TensorFlow, Theano)

1.3.4 1.4 ONNX Runtime

Version: 1.15+

Purpose: Cross-platform model inference

Usage: - Deploy models across frameworks - Optimize inference performance - Hardware acceleration (CPU, GPU, NPU)

Installation:

```
pip install onnxruntime==1.15.0 onnxruntime-gpu==1.15.0
```

1.4 2. Classical Machine Learning

1.4.1 2.1 Scikit-learn

Version: 1.3+

Purpose: Machine learning for Python

Usage: - Random Forest, SVM, Decision Trees - Feature selection and engineering - Model evaluation metrics - Preprocessing utilities

Installation:

```
pip install scikit-learn==1.3.0
```

Key Modules: - `sklearn.ensemble`: Ensemble methods - `sklearn.svm`: Support Vector Machines - `sklearn.tree`: Decision Trees - `sklearn.metrics`: Evaluation metrics - `sklearn.preprocessing`: Data preprocessing - `sklearn.model_selection`: Cross-validation

1.4.2 2.2 XGBoost

Version: 2.0+

Purpose: Gradient boosting framework

Usage: - High-performance boosting - Feature importance - Tree visualization

Installation:

```
pip install xgboost==2.0.0
```

1.4.3 2.3 LightGBM

Version: 4.0+

Purpose: Fast gradient boosting

Usage: - Large-scale datasets - Categorical feature support - Efficient memory usage

Installation:

```
pip install lightgbm==4.0.0
```

1.4.4 2.4 CatBoost

Version: 1.2+

Purpose: Gradient boosting with categorical features

Usage: - Native categorical handling - Robust to overfitting - GPU acceleration

Installation:

```
pip install catboost==1.2
```

1.5 3. Network and SDN Tools

1.5.1 3.1 ONOS (Open Network Operating System)

Version: 2.7+ (Trellis)

Purpose: SDN controller for production networks

Usage: - OpenFlow switch control - Network topology management - REST API for integration - Real-time flow statistics

Installation:

```
# Download and install
```

```
wget https://repol.maven.org/maven2/org/onosproject/onos-releases/2.7.0/onos-2.7.0.tar.
```

```
tar xzf onos-2.7.0.tar.gz
```

```
cd onos-2.7.0
```

```
./bin/onos-service start
```

Key Features: - Carrier-grade SDN controller - High availability and scalability - Intent-based networking - Service applications

1.5.2 3.2 OpenDaylight

Version: Argon (latest)

Purpose: Modular SDN controller

Usage: - Multi-protocol southbound (OpenFlow, NETCONF, BGP) - Flexible north-bound REST APIs - Programmable network services

Installation:

```
# Download and run
```

```
wget https://nexus.opendaylight.org/content/repositories/opendaylight.release/org/opend
```

```
unzip karaf-0.18.0.zip
```

```
cd karaf-0.18.0
```

```
./bin/karaf
```

1.5.3 3.3 Ryu

Version: 4.34+

Purpose: Python-based SDN controller

Usage: - Lightweight controller - Python API for app development - OpenFlow protocol support

Installation:

```
pip install ryu==4.34
```

Usage Example:

```
from ryu.base import app_manager
```

```
from ryu.controller import ofp_event
```

```

from ryu.controller.handler import set_ev_cls

class SimpleSwitch(app_manager.RyuApp):
    def __init__(self, *args, **kwargs):
        super(SimpleSwitch, self).__init__(*args, **kwargs)

    @set_ev_cls(ofp_event.EventOFPPacketIn, MAIN_DISPATCHER)
    def packet_in_handler(self, ev):
        # Handle packet-in events
        pass

```

1.5.4 3.4 Mininet

Version: 2.3+

Purpose: Network emulator for SDN prototyping

Usage: - Virtual network creation - Topology design and testing - OpenFlow switch emulation - Custom topologies in Python

Installation:

```

sudo apt-get install mininet
# Or from source
git clone https://github.com/mininet/mininet
cd mininet
sudo util/install.sh -a

```

Usage Example:

```

from mininet.topo import Topo
from mininet.net import Mininet
from mininet.cli import CLI

class CustomTopo(Topo):
    def build(self):
        # Add switches
        s1 = self.addSwitch('s1')
        # Add hosts
        h1 = self.addHost('h1')
        h2 = self.addHost('h2')
        # Add links
        self.addLink(h1, s1)
        self.addLink(h2, s1)

topo = CustomTopo()
net = Mininet(topo=topo)
net.start()
CLI(net)
net.stop()

```

1.5.5 3.5 NS-3 (Network Simulator 3)

Version: 3.38+

Purpose: Discrete-event network simulator

Usage: - Large-scale network simulation - Protocol testing - Performance evaluation

Installation:

```
git clone https://gitlab.com/nsnam/ns-3-dev.git
cd ns-3-dev
./ns3 configure --enable-examples
./ns3 build
```

1.5.6 3.6 OMNET++

Version: 6.0+

Purpose: Component-based discrete event simulator

Usage: - Network modeling - Protocol simulation - Visualization

Installation: Download from <https://omnetpp.org/>

1.6 4. Data Processing and Analysis

1.6.1 4.1 Pandas

Version: 2.0+

Purpose: Data manipulation and analysis

Usage: - DataFrame operations - CSV/JSON parsing - Time-series analysis - Data cleaning

Installation:

```
pip install pandas==2.0.3
```

1.6.2 4.2 NumPy

Version: 1.25+

Purpose: Numerical computing

Usage: - Array operations - Linear algebra - Fourier transform - Random number generation

Installation:

```
pip install numpy==1.25.0
```

1.6.3 4.3 Apache Spark (PySpark)

Version: 3.4+

Purpose: Distributed data processing

Usage: - Large-scale data processing - Distributed ML (MLlib) - Stream processing

Installation:

```
pip install pyspark==3.4.0
```

1.6.4 4.4 Dask

Version: 2023.7+

Purpose: Parallel computing in Python

Usage: - Parallel pandas operations - Distributed arrays - Task scheduling

Installation:

```
pip install dask==2023.7.0 distributed==2023.7.0
```

1.6.5 4.5 Scapy

Version: 2.5+

Purpose: Packet manipulation and analysis

Usage: - PCAP file reading - Packet crafting - Network sniffing

Installation:

```
pip install scapy==2.5.0
```

Usage:

```
from scapy.all import rdpcap, IP, TCP

packets = rdpcap('traffic.pcap')
for pkt in packets:
    if IP in pkt and TCP in pkt:
        print(f"SRC: {pkt[IP].src}, DST: {pkt[IP].dst}, PORT: {pkt[TCP].dport}")
```

1.7 5. Explainable AI (XAI) Libraries

1.7.1 5.1 SHAP

Version: 0.42+

Purpose: SHapley Additive exPlanations

Usage: - Feature importance - Global and local explanations - Model-agnostic

Installation:


```
pip install shap==0.42.0
```

Usage:

```
import shap

explainer = shap.TreeExplainer(model)
shap_values = explainer.shap_values(X_test)
shap.summary_plot(shap_values, X_test)
```

1.7.2 5.2 LIME

Version: 0.2+

Purpose: Local Interpretable Model-agnostic Explanations

Usage: - Instance-level explanations - Any black-box model

Installation:

```
pip install lime==0.2.0.1
```

Usage:

```
from lime import lime_tabular

explainer = lime_tabular.LimeTabularExplainer(X_train, feature_names=features)
exp = explainer.explain_instance(X_test[0], model.predict_proba, num_features=10)
exp.show_in_notebook()
```

1.7.3 5.3 Captum

Version: 0.6+

Purpose: Model interpretability for PyTorch

Usage: - Attribution algorithms - Layer-wise relevance propagation - Integrated gradients

Installation:

```
pip install captum==0.6.0
```

1.8 6. Distributed Computing and Federated Learning

1.8.1 6.1 TensorFlow Federated

Version: 0.57+

Purpose: Federated learning framework

Usage: - Distributed model training - Privacy-preserving ML - FedAvg implementation

Installation:

```
pip install tensorflow-federated==0.57.0
```

1.8.2 6.2 PySyft

Version: 0.8+

Purpose: Privacy-preserving ML

Usage: - Federated learning - Differential privacy - Secure multi-party computation

Installation:

```
pip install syft==0.8.0
```

1.8.3 6.3 Flower (Flower)

Version: 1.5+

Purpose: Federated learning framework

Usage: - Framework-agnostic FL - Scalable federation - Research and production

Installation:

```
pip install flwr==1.5.0
```

1.9 7. Network Simulation and Testbed**1.9.1 7.1 tcpdump**

Purpose: Packet capture

Installation:

```
sudo apt-get install tcpdump
```

1.9.2 7.2 Wireshark/tshark

Purpose: Network protocol analyzer

Installation:

```
sudo apt-get install wireshark tshark
```

1.9.3 7.3 CICFlowMeter

Purpose: Network traffic flow generator

Usage: Convert PCAP to CSV with flow features

Installation: Download from Canadian Institute for Cybersecurity

1.10 8. Containerization and Orchestration

1.10.1 8.1 Docker

Version: 24+

Purpose: Containerization platform

Usage: - Package IDS as containers - Reproducible environments - Easy deployment

Installation:

```
curl -fsSL https://get.docker.com -o get-docker.sh
sudo sh get-docker.sh
```

1.10.2 8.2 Kubernetes

Version: 1.28+

Purpose: Container orchestration

Usage: - Distributed IDS deployment - Auto-scaling - Service discovery

Installation:

```
curl -LO "https://dl.k8s.io/release/$(curl -L -s https://dl.k8s.io/release/stable.txt)/
sudo install -o root -g root -m 0755 kubectl /usr/local/bin/kubectl
```

1.11 9. Monitoring and Visualization

1.11.1 9.1 Matplotlib

Version: 3.7+

Purpose: Static, animated, and interactive visualizations

Installation:

```
pip install matplotlib==3.7.2
```

1.11.2 9.2 Seaborn

Version: 0.12+

Purpose: Statistical data visualization

Installation:

```
pip install seaborn==0.12.2
```

1.11.3 9.3 Plotly

Version: 5.15+

Purpose: Interactive visualizations

Installation:

```
pip install plotly==5.15.0
```

1.11.4 9.4 Grafana

Version: 10.0+

Purpose: Metrics dashboards

Installation:

```
sudo apt-get install -y grafana
sudo systemctl start grafana-server
```

1.11.5 9.5 Prometheus

Version: 2.45+

Purpose: Time-series monitoring

Installation:

```
wget https://github.com/prometheus/prometheus/releases/download/v2.45.0/prometheus-2.45.0-linux-amd64.tar.gz
tar xvfz prometheus-*.tar.gz
cd prometheus-*
./prometheus --config.file=prometheus.yml
```

1.11.6 9.6 ELK Stack (Elasticsearch, Logstash, Kibana)

Purpose: Log aggregation and analysis

Installation:

Elasticsearch

```
wget https://artifacts.elastic.co/downloads/elasticsearch/elasticsearch-8.9.0-linux-x86_64.tar.gz
tar -xzf elasticsearch-8.9.0-linux-x86_64.tar.gz
cd elasticsearch-8.9.0/
./bin/elasticsearch
```

Kibana

```
wget https://artifacts.elastic.co/downloads/kibana/kibana-8.9.0-linux-x86_64.tar.gz
tar -xzf kibana-8.9.0-linux-x86_64.tar.gz
```

```
cd kibana-8.9.0/  
./bin/kibana
```

1.12 10. Development Tools

1.12.1 10.1 Jupyter Notebook/Lab

Version: Latest

Purpose: Interactive development environment

Installation:

```
pip install jupyter jupyterlab
```

1.12.2 10.2 VS Code

Version: Latest

Purpose: Code editor

Extensions: - Python - Jupyter - Docker - GitLens

1.12.3 10.3 PyCharm

Version: Professional/Community

Purpose: Python IDE

1.13 11. Version Control and Collaboration

1.13.1 11.1 Git

Version: 2.40+

Purpose: Version control

Installation:

```
sudo apt-get install git
```

1.13.2 11.2 GitHub

Purpose: Code hosting and collaboration

Features: - Repository hosting - Issue tracking - Pull requests - GitHub Actions (CI/CD)

1.13.3 11.3 DVC (Data Version Control)

Version: 3.0+

Purpose: Version control for data and models

Installation:

```
pip install dvc==3.0.0
```

1.14 Summary: Complete Environment Setup

1.14.1 Installation Script

```
#!/bin/bash
# Complete environment setup for AI-IDS development

# Update system
sudo apt-get update && sudo apt-get upgrade -y

# Python and pip
sudo apt-get install python3.10 python3-pip -y

# Deep Learning
pip install tensorflow==2.13.0 torch==2.0.1 onnxruntime==1.15.0

# Classical ML
pip install scikit-learn==1.3.0 xgboost==2.0.0 lightgbm==4.0.0

# Data Processing
pip install pandas==2.0.3 numpy==1.25.0 dask==2023.7.0

# XAI
pip install shap==0.42.0 lime==0.2.0.1

# Federated Learning
pip install tensorflow-federated==0.57.0 flwr==1.5.0

# Visualization
pip install matplotlib==3.7.2 seaborn==0.12.2 plotly==5.15.0

# Network Tools
pip install scapy==2.5.0 ryu==4.34

# Development
pip install jupyter jupyterlab ipython

# Docker
curl -fsSL https://get.docker.sh -o get-docker.sh
```

```
sudo sh get-docker.sh  
echo "Environment setup complete!"
```

1.15 Conclusion

This comprehensive toolset provides everything needed for developing, testing, and deploying the AI-based IDS. The combination of deep learning frameworks, classical ML libraries, network simulation tools, XAI libraries, and monitoring solutions enables end-to-end research and development.

Key Stack Highlights: - **Deep Learning:** TensorFlow, PyTorch for model development - **SDN:** ONOS, OpenDaylight, Mininet for network control - **XAI:** SHAP, LIME for explainability - **Federated:** TensorFlow Federated, Flower for distributed learning - **Monitoring:** Grafana, Prometheus, ELK for system observability - **Deployment:** Docker, Kubernetes for production deployment

Next: Refer to `budget.md` for cost allocation and `collaboration.md` for partnership opportunities.